

Memory Management

Thierry Sans

Today's questions

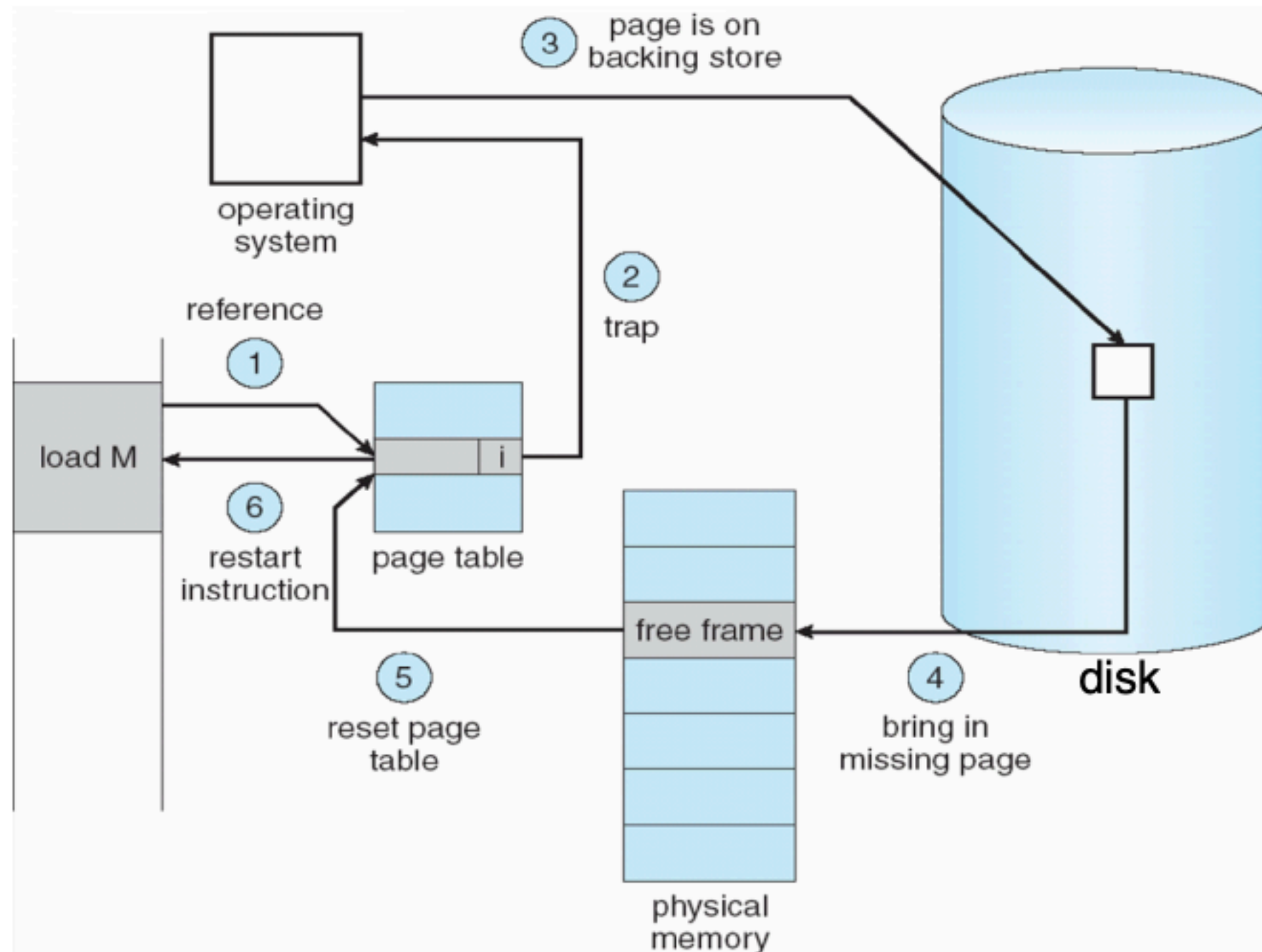
- How to evict pages from memory? (a.k.a when to swap)

Page Replacements Algorithms

Page Replacements Algorithms

(recap) Swapping

➡ Use disk to simulate larger virtual than physical memory



Page Fault and Page Replacement

What happen when there is a page fault?

- ➔ The OS loads the faulted page frame from disk into physical memory

What when there is no physical memory available?

(or the process has reach its limit of maximum page frame allowed)

- ➔ The OS must evict an existing frame (swap) to replace it with the new one

How to determine which page frame should be evicted?

- ➔ The page replacement algorithm (a.k.a page eviction policy) determines which page frame to evict to minimize the fault rate (affecting paging performances)

Page Replacement Algorithms

The goal of the replacement algorithm is to reduce the fault rate by selecting the best victim page to remove

- **FIFO - First In, First Out**
evict the oldest page in the system
- **LRU - Last Recently Used**
evict the page that has not been used for the longest time in the past
- **Second Chance**
an approximation of LRU (more implementable)

➔ Replacement algorithms are evaluated on a reference string by counting the number of page faults

FIFO - First In, First Out (with 3 physical pages)

➔ Evict the oldest page in the system

Access	Hit/Miss	Evict	P0	P1	P2
1	Miss		1		
2	Miss		1	2	
3	Miss		1	2	3
4	Miss	1	4	2	3
1	Miss	2	4	1	3
2	Miss	3	4	1	2
5	Miss	4	5	1	2
1	Hit		5	1	2
2	Hit		5	1	2
3	Miss	1	5	3	2
4	Miss	2	5	3	4
5	Hit		5	3	4

● Total 9 misses

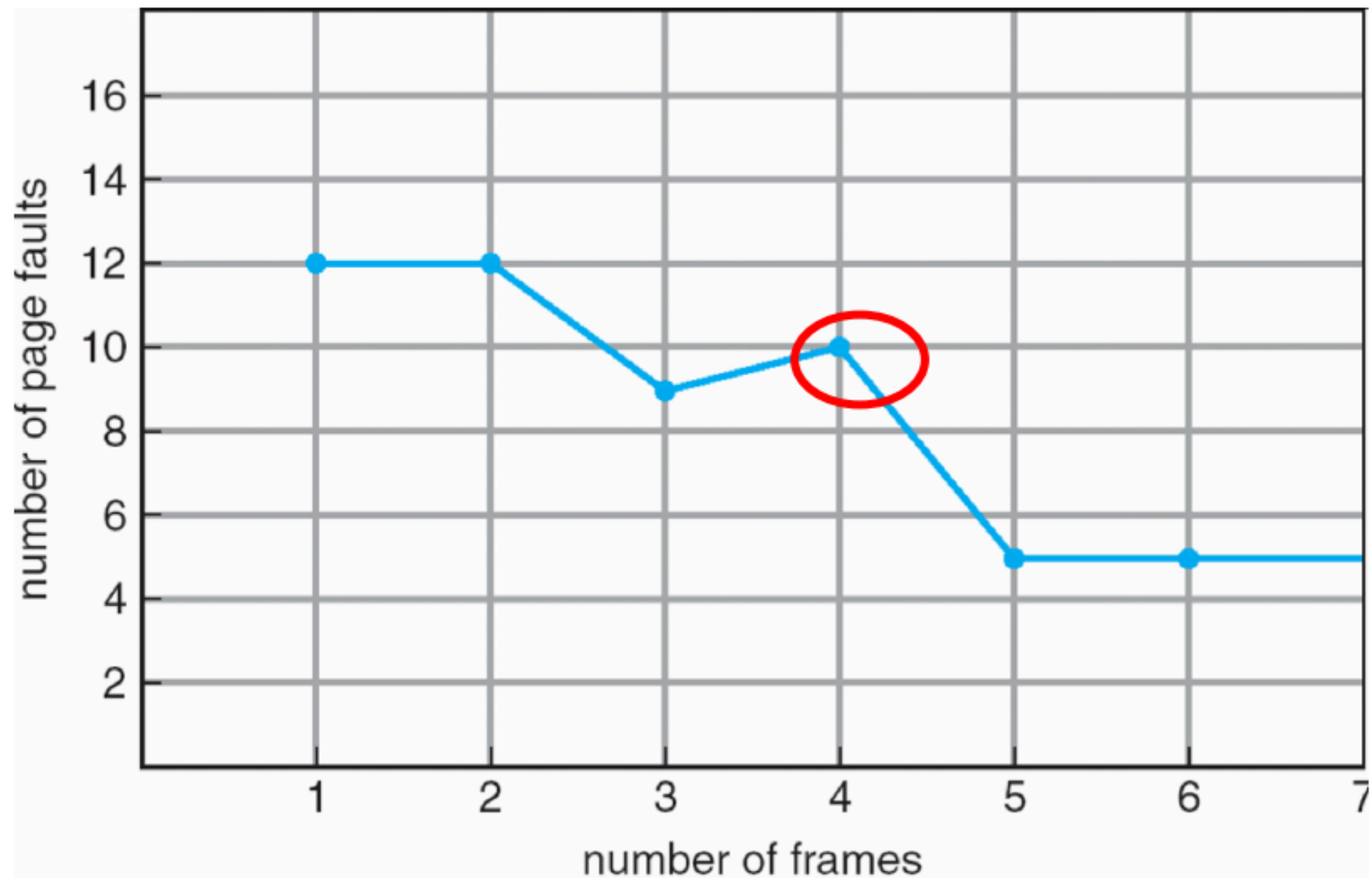
Does having **more physical memory**
automatically means **fewer page faults**?

FIFO - First In, First Out (with 4 physical pages)

Access	Hit/Miss	Evict	P0	P1	P2	P3
1	Miss		1			
2	Miss		1	2		
3	Miss		1	2	3	
4	Miss		1	2	3	4
1	Hit		1	2	3	4
2	Hit		1	2	3	4
5	Miss	1	5	2	3	4
1	Miss	2	5	1	3	4
2	Miss	3	5	1	2	4
3	Miss	4	5	1	2	3
4	Miss	5	4	1	2	3
5	Miss	1	4	5	2	3

● Total 10 misses with 4 physical pages (only 9 with 3 physical pages)

Belady's Anomaly



- More physical memory doesn't always mean fewer faults

Belady's Algorithm

➔ What is optimal if you knew the future?

Access	Hit/Miss	Evict	P0	P1	P2	P3
1	Miss		1			
2	Miss		1	2		
3	Miss		1	2	3	
4	Miss		1	2	3	4
1	Hit		1	2	3	4
2	Hit		1	2	3	4
5	Miss	4	1	2	3	5
1	Hit		1	2	3	5
2	Hit		1	2	3	5
3	Hit		1	2	3	5
4	Miss	1	4	2	3	5
5	Hit		4	2	3	5

● Total 6 misses

Belady's Algorithm

Belady's Algorithm is known (proven) to be the optimal page replacement algorithm

- Problem : it is hard (impossible) to predict the future
- ➔ Belady's algorithm is useful to compare page replacement algorithms with the optimal to gauge room for improvement

LRU - Last Recently Used

➔ Evict the page that has not been used for the longest time in the past

Access	Hit/Miss	Evict	P0	P1	P2	P3
1	Miss		1			
2	Miss		1	2		
3	Miss		1	2	3	
4	Miss		1	2	3	4
1	Hit		1	2	3	4
2	Hit		1	2	3	4
5	Miss	3	1	2	5	4
1	Hit		1	2	5	4
2	Hit		1	2	5	4
3	Miss	4	1	2	5	3
4	Miss	5	1	2	4	3
5	Miss	1	5	2	4	3

● Total 8 misses

How to implement LRU

Idea 1 : stamp the pages with timer value

- On access, stamp the PTE with the timer value
- On miss, scan page table to find oldest counter value

⦿ Problem : would double memory traffic!

Idea 2 : keep doubly-linked list of pages

- On access, move the page to the tail
- On miss, remove the head page

⦿ Problem : again, very expensive!

So, we need to approximate LRU instead

➔ Second Chance page replacement algorithm

Second Chance

Access	Hit/Miss	Evict	P0	P1	P2	P3
1	Miss		1			
2	Miss		1	2		
3	Miss		1	2	3	
4	Miss		1	2	3	4
1	Hit		1*	2	3	4
2	Hit		1*	2*	3	4
5	Miss	3	1	2	5	4
1	Hit		1*	2	5	4
2	Hit		1*	2*	5	4
3	Miss	4	1*	2*	5	3
4	Miss	5	1	2	4	3
5	Miss	3	1	2	4	5

● Total 8 misses

Second Chance implementation

Version 1 : FIFO-like algorithm

➔ use the accessed bit supported by most hardware

Data structure

linked list of pages with two pointers head and tail

Code

- on hit, set the corresponding page's accessed bit to 1
- on miss
 1. while head's accessed bit is 1, set head's accessed bit to 0 and move it to tail
 2. else head's accessed bit is 0, swap the head and move the new page to tail

● Good performances but requires moving pages on every miss

Second Chance implementation

Version 2 : Clock algorithm

➔ use the accessed bit supported by most hardware

Data structure

circular linked list of pages (clock) with one pointer (hand)

Code

- on hit, set the corresponding page's accessed bit to 1
- on miss
 1. while hand's accessed bit is 1, set hand's accessed bit to 0 and move to next page
 2. else if hand's accessed bit is 0, swap the hand's page with the new page and an move next page

● Better performances than fifo-like second chance (no rotation on miss)

Other Replacement Algorithms

Random eviction

- Dirt simple to implement
- Not overly horrible (avoids Belady's anomaly)

LFU (least frequently used) eviction

- Instead of just A bit, count # times each page accessed
- Least frequently accessed must not be very useful (or maybe was just brought in and is about to be used)
- Decay usage counts over time (for pages that fall out of usage)

MFU (most frequently used) algorithm

- Because page with the smallest count was probably just brought in and has yet to be used

➔ Neither LFU nor MFU used very commonly

Linux Paging

- ➡ Global replacement (like most Unix)
 - Modified second-chance clock algorithm
 - Pages age with each pass of the clock hand
 - Pages that are not used for a long time will eventually have a value of zero